EMERGENCY MEDICAL RESPONSE PLATFORM

End-to-End Technical Architecture, Failure Handling & Production Readiness Plan

Version 1.0 | 27 October 2026

Scope: Rapid dispatch, ambulance telemetry, hospital pre-alerts and mass-casualty incidents

Document purpose and safety boundary

This document converts the proposed concept into an engineering plan with explicit state machines, API contracts, reliability controls, security requirements, operational runbooks, testing strategy and corrections to the sample implementation. It is an engineering blueprint, not clinical guidance or certification. A production emergency service must be co-designed with licensed emergency physicians, ambulance operators, hospitals, dispatch authorities, privacy/security counsel and the relevant jurisdiction's emergency communications providers.

Critical constraint: a sub-minute target can describe platform acknowledgement and dispatch initiation, not guaranteed ambulance arrival. GPS accuracy, mobile permissions, network coverage, traffic, fleet availability, operator confirmation and local emergency protocols are outside the platform's complete control. The application must never imply that pressing SOS guarantees a response. It must show the local official emergency number and preserve a manual-call path.

1. Executive summary

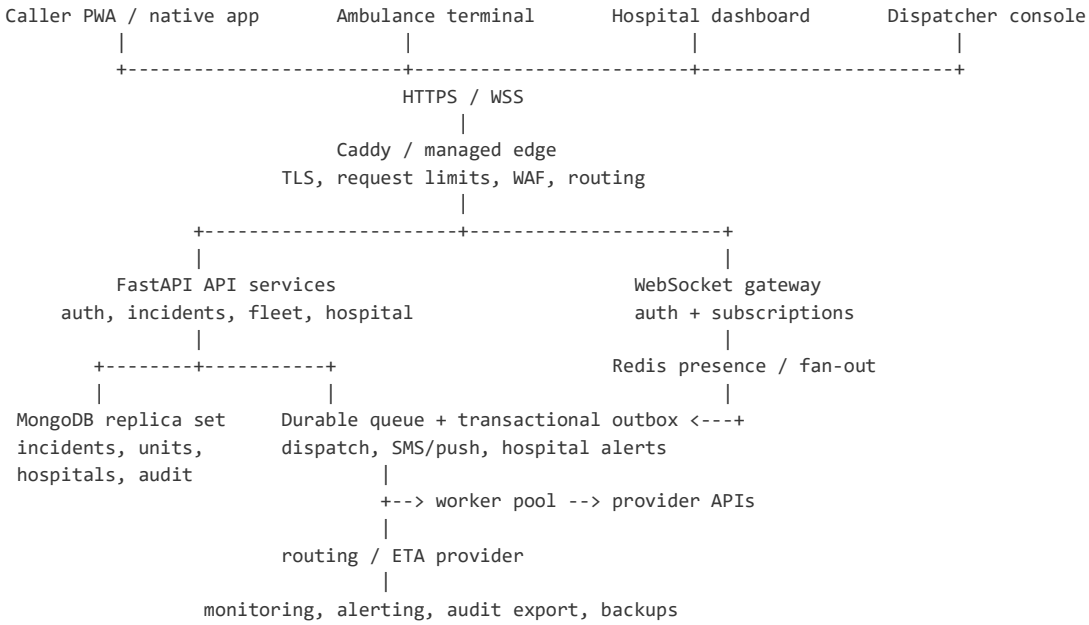
Build a safety-critical, event-driven dispatch platform around a durable incident record and an auditable state machine. Use FastAPI for API services, Next.js/TypeScript for role-specific interfaces, MongoDB geospatial indexes if MongoDB is retained, Redis for ephemeral fan-out and presence, and a durable queue (for example, Celery/RQ/Arq backed by Redis or a managed broker) for dispatch and notification work. Redis Pub/Sub alone is not a durable event bus. Every critical transition must be committed to durable storage before a notification is emitted, and every consumer must be idempotent.

- Caller: one-tap SOS with permission-aware location, a visible location-accuracy indicator, callback details and a fallback to calling the official emergency number.
- Dispatch: candidate ranking by road ETA, equipment/crew capability, availability and jurisdiction—not straight-line distance alone.
- Ambulance: authenticated device/session, heartbeat, assignment acknowledgement, offline status and safe handover between shifts.
- Hospital: capacity and capability are advisory inputs confirmed by authorized staff; no automatic clinical decision or guaranteed bed reservation.
- MCI: incident command, triage tracking, resource requests, hospital distribution and human override with a complete audit trail.
- Reliability: idempotency keys, transactional outbox, leases, compare-and-set assignment, retries with backoff, dead-letter queues and operator fallback.

2. System goals, service objectives and non-goals

Area	Proposed objective	Measurement / caveat
SOS acceptance	p95 under 2 seconds when service and network are healthy	Measure server acknowledgement separately from dispatch and arrival.
Dispatch offer	p95 under 5 seconds after incident persistence	Requires available, connected units and functioning notification paths.
Offer timeout	Configurable; initial 15 seconds may be tested	Timeout must be configurable by jurisdiction and incident class.
Telemetry	p95 under 2 seconds for connected vehicles	Rate-limit, coalesce stale updates and tolerate gaps.
Durability	No acknowledged incident silently lost	Persist incident + outbox before returning success.
Availability	Set SLO after hazard analysis; begin with 99.9% service objective	Emergency operation may require stronger regional redundancy.
MCI	Scale to a tested declared load, not a guessed number	Load test incident bursts, concurrent dispatchers and fleet fan-out.
Clinical decisions	Human-authorized clinical workflows	System may surface information; it must not independently prescribe treatment.

3. Architecture and trust boundaries



Recommended logical modules: Identity & Device Trust; Incident Management; Dispatch Orchestrator; Fleet & Telemetry; Hospital Coordination; MCI Command; Notification Delivery; Audit & Compliance; Reporting. They can initially be a modular monolith with separately scaled WebSocket and worker processes. Do not introduce many microservices before operational ownership and failure boundaries are clear.

4. Key design corrections to the supplied proposal

Issue in draft	Why it is a bug / risk	Required correction
asyncio.create_task(dispatch_units_task(...))	Task is lost on process restart; exceptions may be unobserved; response can succeed while dispatch never occurs.	Write an outbox event in the same durable transaction as incident creation; worker consumes it with retries.
Redis Pub/Sub treated as durable	Subscribers miss messages while disconnected; no replay or consumer acknowledgements.	Use Pub/Sub only for transient fan-out; durable queue/outbox is source of work.
Candidates queried then notified without reservation	Two simultaneous incidents can assign the same ambulance.	Acquire assignment with atomic compare-and-set / lease and unique active-assignment constraint before offer.
Nearest by $nearSphere only	Straight-line distance can select a unit separated by rivers, highways or traffic.	Use geospatial search to shortlist; rank using road ETA, capability, crew status and policy.
GeoJSON Point has no coordinate validation	MongoDB expects [longitude, latitude]; swapped or out-of-range values can route incorrectly.	Validate exactly two finite coordinates: longitude -180..180, latitude -90..90; reject invalid accuracy.
Caller phone required for SOS	Can block anonymous or privacy-conscious callers and can be mistyped.	Make callback optional where policy allows; provide immediate call fallback and minimal friction.
Device fingerprint used as authentication	Fingerprinting is unstable, spoofable and privacy-invasive.	Use short-lived anonymous session token, abuse controls, device attestation only where justified, and no fingerprint as sole trust factor.
No idempotency	Retries can create duplicate incidents and duplicate	Require Idempotency-Key; persist request hash and replay

	dispatches.	the original result for matching requests.
No explicit incident state machine	Concurrent updates can regress or skip states.	Enforce allowed transitions using versioned compare-and-set updates.
MCI unit formula is underspecified	A hard cap of 5/20 can under-resource a major event or over-dispatch a small one.	Use incident command estimates and configurable resource matrix; system recommends, authorized command confirms.
Hospital auto-reservation assumptions	Live capacity may be stale and clinical needs can change.	Send pre-alert and request acknowledgement; record capacity timestamp and authorized confirmation.
SMS fallback described as automatic browser SMS	Browsers generally cannot silently send SMS; SMS may be unavailable or delayed.	Offer a tap-to-call/tap-to-SMS fallback or use an approved gateway; clearly show delivery status.
Immutable MongoDB log implied	A normal collection is not intrinsically immutable.	Use append-only permissions, hash chaining or signed exports, retention controls and external/WORM backups.
Motor dependency	Motor has been deprecated in favor of PyMongo's async API in modern MongoDB Python guidance.	For a new build, evaluate PyMongo Async; pin supported versions and test driver behavior.
Emergency siren/audio assumption	Browsers can block autoplay and notifications can be muted.	Require visible in-app alert, vibration where supported, sound after user interaction, and redundant channels.

5. Recommended technology choices

Layer	Recommended choice	Notes
Web clients	Next.js + TypeScript; PWA where appropriate	Caller PWA is not equivalent to a native app for background location or SMS capabilities.
API	FastAPI + Pydantic v2 + Uvicorn/ASGI	Use lifespan-managed clients, dependency injection, typed settings and consistent error schemas.
Database	MongoDB replica set with 2dsphere indexes OR PostgreSQL/PostGIS	Choose one source of truth. MongoDB is viable for GeoJSON; PostGIS is strong for relational constraints and reporting.
Mongo driver	PyMongo Async if MongoDB remains	Avoid starting a new project on deprecated Motor without a documented compatibility reason.
Durable work	Transactional outbox + durable queue	Choose broker based on operations skill; queue messages are at-least-once, so handlers must be idempotent.
Realtime	WebSockets + Redis for presence/fan-out	Re-authenticate, authorize each subscription, heartbeat, reconnect with backoff and replay current state via REST.
Maps / ETA	OSRM or licensed routing API	Do not assume emergency signal priority or traffic overrides unless formally integrated and legally supported.
Edge	Caddy or managed ingress	TLS, body limits, rate limits, request

		IDs; do not rely on edge controls alone.
Observability	OpenTelemetry, structured logs, metrics, traces	Never put phone numbers, exact coordinates or patient vitals in routine logs.
Secrets	Cloud secret manager / KMS	No secrets in source, frontend bundles, images or logs.

6. Domain model and state machines

6.1 Incident lifecycle

```
RECEIVED -> VALIDATING -> QUEUED -> OFFERING -> ASSIGNED -> EN_ROUTE -> ON_SCENE -> TRANSPORTING ->
HANDED_OVER -> CLOSED
```

Additional terminal or side states: CANCELLED (only under authorized policy), DUPLICATE_LINKED, ESCALATED_MANUAL, NO_RESOURCE, FAILED_REQUIRES_OPERATOR. Do not overwrite history: append transition events and maintain a current-state projection. A cancellation must not silently remove a dispatched unit; it must send a cancellation event and require acknowledgement where applicable.

6.2 Ambulance lifecycle

```
OFF_DUTY -> AVAILABLE -> OFFERED -> DISPATCHED -> EN_ROUTE -> ON_SCENE -> TRANSPORTING -> HANDOVER ->
AVAILABLE
```

An OFFERED unit is temporarily leased to one offer. The lease expires unless accepted; lease expiry triggers a compare-and-set release and next-candidate offer. Do not set a unit AVAILABLE solely because its WebSocket disconnected. Mark presence stale, preserve assignment state, and notify dispatch.

6.3 Required entities

Entity	Important fields	Constraints
Incident	id, idempotency_key, request_hash, category, location, accuracy_m, status, severity, created_at, version	Unique idempotency key per caller/session scope; validated coordinates.
DispatchOffer	id, incident_id, unit_id, offer_number, offered_at, expires_at, state	Unique active offer per unit; unique offer number per incident.
Ambulance	id, operator_id, status, last_seen_at, location, capabilities, crew_state, version	Location timestamp and accuracy; one active assignment max.
Assignment	id, incident_id, unit_id, accepted_at, released_at, state	Atomic unit claim; no conflicting active assignments.
Hospital	id, location, capabilities, status, capacity_snapshot_at, acknowledged_at	Capacity values are time-stamped and confirmed by authorized staff.
TriageRecord	id, incident_id, patient_ref, triage_tag, vitals, author_id, recorded_at, version	Strict access controls; provenance and correction history.
OutboxEvent	id, aggregate_id, event_type, payload, created_at, published_at, attempts	Durable, idempotent event processing.
AuditEvent	id, actor, action, target, occurred_at, request_id, previous_hash, hash	Append-only service role; export integrity verification.

7. API contracts

Use versioned JSON APIs, a shared error envelope, request IDs, pagination, explicit timestamps in UTC and idempotency on mutating endpoints.

Endpoint	Purpose	Access / protections
POST /api/v1/emergency/sos	Create incident and durable dispatch request	Caller session; Idempotency-Key; rate limit; schema validation.
GET /api/v1/incidents/{id}	Current status and caller-safe details	Opaque ID plus scoped authorization; do not expose other callers' records.
POST /api/v1/incidents/{id}/cancel-request	Request cancellation	Caller/dispatcher policy; dispatch acknowledgement workflow.
POST /api/v1/ambulances/{id}/telemetry	Fallback telemetry ingestion	Vehicle device credential; sequence number; timestamp and bounds validation.
WS /ws/ambulance	Offers, acknowledgements, assignment updates	Mutual device auth; per-unit authorization; heartbeat and replay.
POST /api/v1/dispatch/offers/{id}/accept	Accept offer	Authenticated unit; atomic lease check; idempotent response.
POST /api/v1/dispatch/offers/{id}/decline	Decline offer with reason	Authenticated unit; state transition guard.
POST /api/v1/triage	Create/update triage record	Role-based permission; field validation; full audit.
POST /api/v1/hospitals/{id}/prealerts	Send pre-alert	Dispatch/clinical role; minimize data; acknowledgement tracking.
POST /api/v1/mci/{id}/resources	Request/approve MCI resources	Incident command role; approval and audit.
GET /api/v1/health/live	Process liveness	No dependency details.
GET /api/v1/health/ready	Readiness of required dependencies	Internal/edge restricted; safe dependency status only.

7.1 SOS request schema recommendations

```
class GeoJSONPoint(BaseModel):
    type: Literal["Point"] = "Point"
    coordinates: tuple[float, float]  # [longitude, latitude]
    # Add validators: finite numbers; lon in [-180, 180], lat in [-90, 90].

class IncidentCreate(BaseModel):
    location: GeoJSONPoint
    location_accuracy_m: float | None = Field(default=None, ge=0, le=10000)
    caller_phone: str | None = None
    landmark: str | None = Field(default=None, max_length=250)
    incident_category: Literal["medical", "trauma", "cardiac", "fire_collapse", "other"]
    estimated_casualties: int = Field(default=1, ge=1, le=500)
    # casualty_scale should be derived or checked against authorized triage/dispatcher input.
```

Use Pydantic validators for normalized phone numbers, finite floats, maximum payload size, allowed fields and cross-field rules. Do not accept client-provided ambulance IDs, hospital capacity or privileged severity as trusted facts.

7.2 Standard error envelope

```
{
  "error": {
    "code": "LOCATION_INVALID",
    "message": "A valid location is required.",
    "request_id": "server-generated-correlation-id",
    "retryable": false
  }
}
```

Avoid returning stack traces, database identifiers, secrets or provider responses. Define stable codes such as RATE_LIMITED, IDEMPOTENCY_CONFLICT, OFFER_EXPIRED, RESOURCE_UNAVAILABLE and DEPENDENCY_UNAVAILABLE.

8. Dispatch algorithm: safe, race-free and recoverable

1. Validate request, caller session, coordinates, accuracy, idempotency key and abuse signals.
2. Persist the incident and an IncidentCreated outbox record atomically. Return a durable acknowledgement with incident ID and clear next-step messaging.
3. Worker claims the outbox event with a lease. Duplicate delivery is expected; deduplicate by event ID.
4. Shortlist available ambulances using a 2dsphere query with a bounded radius and fresh location timestamp.
5. Filter by service area, vehicle capability, crew qualification, shift status, equipment and active assignments.
6. Ask routing provider for road ETA where available; rank candidates using ETA plus policy constraints. If routing is down, use a clearly marked fallback estimate and dispatcher review.
7. Atomically claim a unit with compare-and-set: status must still be AVAILABLE, version must match, and no active assignment may exist. Create a time-limited offer.
8. Send the offer over WebSocket and a backup push channel. Persist delivery attempts. Do not assume a WebSocket send means the driver saw the offer.
9. On ACCEPT, atomically validate offer state and expiry, convert lease to assignment, record actor/time, then notify caller and dispatch console.
10. On DECLINE, timeout or delivery failure, release the lease and offer the next candidate. Apply bounded retries, then escalate to a human dispatcher.
11. Notify hospital only after destination and patient/triage data are sufficiently confirmed; request acknowledgement and keep delivery state visible.
12. Continue telemetry and state transitions. Reconcile stale units, unacknowledged offers and stuck incidents through scheduled sweeper jobs.

Offer sequencing must be policy-configurable. Parallel offers can reduce latency but risk duplicate responses and resource contention; use them only with explicit reservation leases and cancellation/stand-down handling.

8.1 Transactional outbox pattern

```
# Conceptual transaction (adapt to chosen database):
1. insert incident
2. insert outbox event IncidentCreated with unique event_id
3. commit
4. worker reads unpublished outbox rows/documents
5. publish to durable queue; mark published
6. consumer processes idempotently and records completion
```

If the database and queue do not share a transaction, the outbox closes the failure gap between committing an incident and publishing its work. The publisher may publish twice; consumers must deduplicate.

9. Location, routing and telemetry edge cases

- Permission denied: allow manual landmark/address and immediate official-call action; do not block emergency contact on GPS.
- GPS unavailable: use last-known location only with timestamp and explicit stale marker; never present it as current.
- Poor accuracy: show accuracy radius and request confirmation; dispatch can still proceed with a warning if policy permits.
- Coordinate order swapped: validate ranges and add tests using known landmarks; GeoJSON is longitude first, latitude second.
- Mocked or spoofed location: treat coordinates as untrusted; use movement plausibility checks and operator confirmation, not silent rejection of a genuine emergency.
- Urban canyon / indoor: support entrance, floor, building, gate and landmark notes; do not claim sub-meter accuracy from a browser by default.
- Stale vehicle GPS: reject out-of-order sequence numbers; retain last update time; mark unit stale after configured threshold.
- Telemetry burst: coalesce location updates, cap frequency and preserve important state transitions separately.

- Map provider outage: retain last-known route/ETA with timestamp, switch provider only if contract and licensing allow, and escalate to dispatcher.
- Clock skew: server timestamps are authoritative for state transitions; device timestamps are metadata and checked for skew.
- Battery/data saver: reconnect with exponential backoff and jitter; send compact heartbeat; provide voice/radio/phone fallback.

10. Mass-casualty incident (MCI) operations

Treat automated clustering as a suggestion, not a declaration. Several callers near one another may describe the same event, unrelated events, or duplicate calls. Use time-and-distance clustering as a signal for dispatcher review, correlate caller reports, and maintain an explicit parent incident with linked reports.

10.1 MCI workflow

13. Trigger: dispatcher/incident commander declaration, or an automated candidate alert based on multiple reports and configurable confidence.
14. Create an MCI parent incident and link child calls; preserve each caller's report and original timestamps.
15. Assign an incident commander and establish command roles, staging areas, communication channel and operational period.
16. Record triage counts by category with timestamp and provenance; avoid treating estimates as confirmed patient counts.
17. Request resources by capability (ALS/BLS, ventilator, bariatric, burn, neonatal, rescue) rather than only raw vehicle count.
18. Offer resources from municipal/private operators under pre-agreed mutual-aid rules, jurisdiction and credential checks.
19. Maintain a resource board: requested, offered, accepted, en route, staged, assigned, released and unavailable.
20. Coordinate hospital distribution using capability, current capacity freshness, travel time, specialty needs and authority protocols.
21. Require human approval for destinations, diversion, triage policy exceptions and high-impact resource allocation.
22. Close the MCI only after command authorization, reconciliation of patients/resources and completion of after-action records.

10.2 Resource calculation

Do not ship a universal formula such as casualties × 1.25 or a fixed cap of 20 as the dispatch truth. Resource needs depend on injury severity, scene access, transport distance, crew capability, extrication requirements, weather and local protocol. The system may calculate a configurable recommendation with a visible assumption set, but the incident commander confirms, changes or rejects it. Keep recommendation, decision, author and reason in the audit trail.

10.3 Duplicate reports and false MCI signals

- Cluster by configurable spatial radius and time window, but also compare incident category, landmarks and dispatcher notes.
- Show confidence and supporting reports; never auto-merge reports irreversibly.
- Allow authorized staff to split a wrongly merged incident and preserve lineage.
- Rate-limit automated escalation to prevent a caller flood from exhausting all fleet resources.
- Use human review and a safe regional escalation path if the clustering service is unavailable.

11. Hospital pre-alert and clinical-data safeguards

- Pre-alerts must be acknowledged, time-stamped and retried; delivery to an API is not equivalent to a clinician seeing it.
- Send the minimum necessary patient information for care coordination; encrypt in transit and at rest.

- Vitals include units, measured_at, measured_by, method and validity flags. Reject impossible values but preserve corrections with provenance.
- Black/deceased triage categories and clinical labels must follow the locally approved triage protocol; terminology varies by jurisdiction.
- Blood bank, CT, trauma bay and specialist notifications should be recommendations/workflow tasks, not automatic reservations unless the hospital explicitly supports that workflow.
- Capacity must include source and freshness. If capacity is stale, display unknown and require confirmation.
- Support hospital decline, diversion, no acknowledgement, system outage, destination change and en-route deterioration.
- Do not let a model or rule engine independently diagnose, triage, choose treatment or override a clinician/dispatcher.

12. Security, privacy and access control

Control	Implementation requirement
Authentication	Short-lived access tokens; refresh rotation; secure, HttpOnly, SameSite cookies for browser sessions; CSRF protection where cookie-authenticated mutations exist.
Device identity	Per-vehicle credentials/certificates with revocation and rotation; bind identity to operator/unit; never trust a client-supplied ambulance_id alone.
Authorization	RBAC plus resource/tenant scoping: caller, paramedic, dispatcher, hospital clinician, MCI commander, admin, auditor.
Transport/storage	TLS everywhere; database encryption; managed keys; least-privilege service accounts.
Secrets	Secret manager, rotation, no secrets in frontend or logs; redact provider tokens.
Abuse prevention	Per-IP/session/device limits, bot mitigation, replay protection, anomaly detection and a dispatcher-safe surge mode.
Privacy	Purpose limitation, data minimization, retention schedules, access logging, data-subject processes and jurisdiction-specific review.
Audit	Log access to sensitive records; append-only transition records; signed/hash-chained exports; restricted deletion and export permissions.
WebSockets	Authenticate at connect; authorize every subscription/action; enforce message schemas, size limits, origin checks and connection quotas.
Uploads	If ID/medical images are allowed: MIME sniffing, size limits, malware scanning, private object storage and short-lived signed URLs.
Incident response	Credential revocation, breach runbook, security contact, forensic preservation and post-incident review.

Avoid collecting browser fingerprints as a substitute for authentication. A device fingerprint can be inaccurate and privacy-invasive, and can be spoofed. Emergency access must be friction-light but not permission-free for fleet and clinical data.

13. Reliability and failure-mode matrix

Failure	Detection	Required behavior	Recovery / owner
API process crash	Health check, restart count	Committed incident remains durable; queue retries	Platform on-call; worker replay
Database unavailable	Connection/latency alerts	Return safe unavailable	Failover; restore;

		message; do not claim dispatch accepted unless durable write succeeded	dispatcher phone/radio
Queue unavailable	Outbox backlog metric	Persist incident/outbox if DB healthy; backlog alarm; no silent drop	Restore broker; replay outbox
Redis unavailable	Connectivity and fan-out errors	REST and durable dispatch continue; reconnect WebSockets; avoid relying on Redis for source of truth	Restore Redis; rebuild presence
WebSocket disconnect	Heartbeat timeout	Push/alternate channel; unit marked stale; preserve assignment	Reconnect; dispatcher confirms
Push/SMS provider outage	Delivery receipts and error rates	Retry with backoff; alternate provider/channel; operator queue	Provider failover
Routing API timeout	Timeout/error ratio	Use last valid ETA marked stale or safe fallback; dispatch review	Retry/circuit breaker
No ambulance available	Empty candidate list	Escalate to regional operator; display official call option	Mutual aid / neighboring region
Unit accepts after expiry	Atomic state conflict	Reject stale acceptance and return current state; do not create duplicate assignment	Offer next unit; dispatcher handles late arrival
Two incidents claim same unit	CAS/unique constraint conflict	Only one claim wins; loser tries next candidate	Automatic retry
Duplicate SOS retry	Idempotency key match	Return same incident ID; do not duplicate dispatch	No action
False GPS / stale coordinates	Accuracy, timestamps, plausibility	Flag location uncertainty; request confirmation; do not silently discard emergencies	Dispatcher verification
Hospital does not acknowledge	Timer and delivery state	Retry, call/dispatch escalation, alternate destination only by authorized protocol	Hospital liaison/dispatcher
MCI overload	Queue age, active incident count	Prioritize by approved policy; shed noncritical analytics, not emergency dispatch	Incident command and capacity plan
Regional power/network outage	Synthetic checks and carrier status	Fallback to official emergency number, radio/telephone and approved SMS gateway	Business continuity plan
Clock skew / bad timestamps	Skew metrics	Server transition time is authoritative; preserve device time separately	NTP monitoring
Bad deployment/schema migration	Canary and error budget	Backward-compatible migrations; rollback; feature flags	Release manager
Ransomware/data corruption	Integrity checks, immutable backups	Isolate affected systems; operate manual dispatch plan	Security incident commander

14. Retry, timeout, idempotency and concurrency rules

- Set explicit connect/read/write timeouts for every external dependency. Never wait indefinitely on routing, SMS, push or hospital systems.
- Use bounded exponential backoff with jitter for transient failures; do not retry validation errors or permanent authorization failures.
- Use circuit breakers for failing third-party providers and a bulkhead so a slow provider cannot consume all API workers.
- Idempotency scope should include authenticated caller/session plus key. Store request hash, status and response; same key with different body returns conflict.
- Use database atomic updates/version fields for state transitions. Validate current state and expected version in the update filter.
- Use a unique constraint/index for one active assignment per ambulance, implemented in a database-appropriate way.
- Every offer has a server-generated ID, expiry, sequence number and terminal state. Acceptance/decline must be idempotent.
- Run reconciliation jobs for stuck outbox items, expired leases, unacknowledged alerts, stale fleet states and incidents with impossible transitions.

15. Observability and operational dashboards

Signal	Suggested metric / alert
SOS intake	request rate, validation failures, p50/p95/p99 latency, durable-write failures
Dispatch	time from persisted incident to first offer; offer acceptance rate; timeout rate; units searched; manual escalation count
Queue	outbox age, oldest pending event, queue depth, retry count, dead-letter count
Fleet	connected units, heartbeat age, stale GPS count, GPS age/accuracy distribution, location update delay
Hospital	pre-alert delivery/acknowledgement latency, stale capacity snapshots, failed destination confirmations
MCI	reports linked, triage-count changes, resource requests outstanding, unacknowledged command actions
Security	auth failures, privilege denials, rate-limit events, unusual export/access volume
Resilience	dependency availability, circuit-breaker state, failover events, recovery time
Data integrity	state transition conflicts, duplicate idempotency hits, audit-chain verification failures

Use synthetic end-to-end checks in a non-emergency test environment. Alert on user-impacting symptoms and queue age, not just CPU. Keep dashboards and logs free of unnecessary patient identifiers and exact coordinates.

16. Deployment, scaling and disaster recovery

23. Separate production, staging and development accounts/projects and credentials.
24. Deploy at least two API instances across failure domains; scale WebSocket gateways separately from HTTP API and workers.
25. Use MongoDB replica set with tested backups and point-in-time recovery if supported by the selected service; verify restore drills.
26. Use managed durable queue/broker or a well-operated broker cluster; monitor disk, consumer lag and replication.
27. Run database migrations as a controlled release step with expand-migrate-contract compatibility.
28. Use canary/blue-green deployment, feature flags and rollback. Avoid breaking changes to live ambulance clients.

29. Define RPO/RTO based on hazard analysis. For emergency-critical data, test regional recovery rather than merely documenting it.
30. Keep an offline/manual dispatch plan with contact lists, radio/phone procedures, printed or exportable roster and authority-approved fallback.
31. Conduct quarterly failover exercises and after-action reviews; update runbooks after every exercise.

Do not state an availability percentage or requests-per-second claim as achieved until load tests, dependency limits, recovery exercises and operational staffing demonstrate it.

17. Testing and verification plan

Test category	Must-cover cases
Unit	coordinate validation, phone normalization, state transitions, ETA ranking, MCI policy calculations, authorization checks
Property-based	random coordinates, malformed GeoJSON, numeric extremes, duplicate events, invalid transition sequences
Concurrency	two incidents claim same unit; duplicate accept; accept vs expiry race; dispatcher reassignment vs driver acceptance
Integration	Mongo indexes, outbox replay, queue redelivery, Redis loss, provider timeouts, hospital acknowledgements
Contract	API schemas, event schemas, backwards compatibility for ambulance app versions
Load	SOS bursts, MCI fan-out, thousands of telemetry updates, WebSocket reconnect storm, slow downstream provider
Chaos / fault injection	kill API during dispatch, broker outage, Redis restart, database failover, packet loss, DNS/provider outage
Security	IDOR, CSRF, token replay, privilege escalation, WebSocket authorization, injection, rate-limit bypass, secrets scan
Usability/accessibility	large SOS target, screen reader, low bandwidth, low battery, localization, color-blind-safe triage labels
Operational drills	no resources, incorrect destination, MCI declaration, hospital unreachable, regional outage, credential compromise
Clinical simulation	authorized clinicians validate triage fields, pre-alert content, acknowledgement workflow and escalation policy

Use only simulated incidents in testing. Never test dispatch or emergency notifications against live fleets or hospitals without explicit written authorization and a coordinated exercise plan.

18. Release gates and acceptance criteria

- An SOS acknowledgement is returned only after the incident and outbox event are durably committed.
- Repeated requests with the same idempotency key do not create a second incident or dispatch.
- Race tests prove that one ambulance cannot be actively assigned to two incidents.
- Expired or stale offers cannot be accepted; late acceptance produces a safe, explicit conflict.
- Database/queue/Redis/provider failures trigger observable, tested fallback behavior.
- Every state transition includes actor, server timestamp, request/correlation ID and prior/new state.
- Sensitive patient and location data are authorized, encrypted, redacted from logs and governed by retention rules.
- Manual dispatcher escalation works when no unit is available or automated dispatch is unhealthy.
- Hospital capacity freshness and acknowledgement state are visible; no stale snapshot is presented as current.

- MCI declarations, merges, splits and resource overrides are audited and human-authorized.
- Backup restoration, rollback, regional recovery and offline operational drills pass before launch.
- Clinical, legal, privacy, ambulance operator and emergency authority sign-offs are recorded.

19. Implementation roadmap

Phase	Deliverables	Exit gate
0. Governance & hazard analysis	Stakeholders, operating jurisdiction, emergency authority, clinical safety case, privacy assessment, fallback process	Hazards, ownership and approval path documented
1. Reliable core	Incident schema, idempotent SOS, state machine, outbox, operator queue, audit events	No lost acknowledged incident in fault-injection tests
2. Fleet dispatch	Vehicle identity, telemetry, leases, accept/decline, stale-unit recovery	Concurrency and reconnect tests pass
3. Caller experience	Location permission UX, accuracy display, call fallback, incident status	Usability/accessibility tests pass
4. Hospital coordination	Pre-alert, acknowledgement, capacity freshness, destination change	Partner integration test signed off
5. MCI mode	Parent/child incidents, resource board, triage counts, command roles	Tabletop and simulated field exercise passed
6. Hardening	Load, penetration, chaos, disaster recovery, training, runbooks	Release gates passed
7. Controlled pilot	Limited region, staffed operations, explicit fallback, monitored metrics	Formal go/no-go by accountable authorities

20. Operational runbooks

20.1 No candidate ambulance

32. Mark incident NO_RESOURCE / ESCALATED_MANUAL without closing it.
33. Alert regional dispatch and show the official emergency calling option to the caller.
34. Attempt authorized mutual-aid search and record each attempt.
35. Keep caller status honest; do not show a fabricated ETA.
36. Escalate to the duty supervisor after the configured threshold.

20.2 Dispatch queue backlog

37. Page platform on-call when oldest pending event exceeds the SLO threshold.
38. Check database health, outbox publisher, broker, consumer lag and provider status.
39. Scale workers only after confirming downstream capacity and avoiding duplicate sends.
40. Keep manual dispatch active; do not disable idempotency or state checks to clear backlog.
41. Reconcile every pending incident after recovery.

20.3 Hospital unavailable

42. Mark hospital integration unavailable and timestamp last successful acknowledgement.
43. Notify dispatch; use approved phone/radio protocol for clinical coordination.
44. Do not automatically redirect a patient without authorized clinical/dispatch policy.
45. Record destination decision, authority and time; retry integration safely.

20.4 Suspected compromised vehicle device

46. Revoke device credentials and block new sessions.
47. Preserve logs and assignment history; notify dispatch to verify the unit by trusted channel.
48. Reassign only through the normal atomic assignment workflow.

49. Rotate credentials and complete incident review before re-enrollment.

21. Data retention, governance and compliance

Before deployment, map the system to applicable Indian and local requirements for emergency communications, health information, personal data, telecom/SMS, medical records, ambulance operations, accessibility and public procurement. The applicable rules depend on the operating entity and jurisdiction; obtain qualified legal review rather than assuming a generic checklist is sufficient.

- Define retention periods by data class: incident metadata, location history, clinical data, audit events, delivery receipts and support logs.
- Restrict access to exact location and clinical records by role and operational need; record all access and exports.
- Separate analytics identifiers from directly identifying records; use aggregation for routine dashboards.
- Provide correction workflows that preserve original values and show who corrected them.
- Document data processors, routing/map providers, SMS/push vendors, subprocessors, data residency and breach notification obligations.
- Test deletion/retention jobs against legal holds and incident investigations; avoid deleting audit evidence accidentally.

22. Revised implementation pseudocode

```
async def create_sos(payload, idempotency_key, caller_context, db):
    validate_coordinates(payload.location)
    validate_accuracy(payload.location_accuracy_m)
    enforce_rate_limit(caller_context)

    # In one database transaction where supported:
    existing = await find_idempotency_record(caller_context.scope, idempotency_key)
    if existing:
        if existing.request_hash != hash_request(payload):
            raise IdempotencyConflict()
        return existing.original_response

    incident = build_incident(payload, status="QUEUED")
    await persist_incident_and_outbox_atomically(
        incident=incident,
        event_type="IncidentCreated",
        idempotency_key=idempotency_key,
        request_hash=hash_request(payload),
    )
    return {"incident_id": incident.id, "status": "received",
            "message": "Incident received; dispatch status will update shortly."}


async def dispatch_worker(event):
    if await event_already_processed(event.id):
        return

    incident = await load_current_incident(event.aggregate_id)
    if incident.is_terminal:
        await mark_event_processed(event.id)
        return

    candidates = await shortlist_fresh_available_units(incident.location)
    ranked = await rank_by_road_eta_capability_and_policy(candidates, incident)

    for unit in ranked:
        offer = await atomically_claim_unit_and_create_offer(
            incident_id=incident.id,
            unit_id=unit.id,
            expires_in_seconds=config.offer_timeout_seconds,
        )
```

```
            if not offer:
                continue  # another incident won the race

            result = await send_offer_with_timeout_and_fallback(offer)
            await record_delivery_result(offer.id, result)
            # Await accept/decline/timeout event; do not block a worker indefinitely.
            return

        await mark_incident_escalated_manual(incident.id)
        await notify_dispatch_operator(incident.id)
```

This is intentionally pseudocode: exact transaction syntax and indexes depend on whether MongoDB or PostgreSQL is selected. The critical properties are durable intent, atomic claims, idempotent consumers, explicit state transitions and a staffed fallback—not the particular framework syntax.

23. Final engineering recommendations

- Start with a modular monolith plus durable workers; split services only where scale or isolation proves the need.
- Keep a single authoritative store for incident and assignment state. Redis is a cache/presence/fan-out layer, not the source of truth.
- Replace fire-and-forget asyncio tasks with a transactional outbox and durable workers.
- Use atomic leases and database constraints to eliminate double-dispatch races.
- Rank by road ETA and capabilities, not only geospatial proximity.
- Treat automated MCI detection and hospital capacity as decision support; keep authorized human command in control.
- Make official-call and manual-dispatch fallback visible from day one.
- Do a formal hazard analysis, clinical safety review, threat model, privacy assessment and regional operational exercise before live deployment.

Appendix A — Production checklist

☐ Operating jurisdiction and official emergency-number flow approved

☐ Clinical triage schema and terminology approved by licensed clinical authority

☐ Ambulance operators and hospitals have signed integration/operational agreements

☐ Incident state machine and transition table implemented

☐ Idempotency and request hashing implemented

☐ Transactional outbox and durable queue implemented

☐ Atomic assignment lease and active-assignment constraint implemented

☐ GeoJSON coordinate validation and known-point tests implemented

☐ ETA fallback and provider circuit breakers implemented

☐ WebSocket authentication, authorization, replay and reconnect implemented

☐ Push/SMS provider delivery receipts and alternate channel implemented

☐ Hospital acknowledgement and capacity freshness implemented

☐ MCI parent-child linking, merge/split and command audit implemented

☐ Audit append-only controls and integrity verification implemented

☐ Data retention and access review completed

☐ Metrics, tracing, alerting and on-call rotation active

☐ Backup restoration and regional failover tested

☐ Load, security, concurrency and chaos tests passed

☐ Offline/manual dispatch drill completed

☐ Pilot go/no-go approval recorded